

File System & Disk Driver Design

Dr. Zizhan CHEN

1. Project Overview

This project implements a lightweight file system (inspired by MINIX FS) together with an IDE hard-disk driver.

File System Features

- Supports core operations: create, open, close, read, write, delete, flush
- Employs block caching (LRU) for efficient data access
- Uses inode caching to further improve performance

Disk Driver Features

- Maintains an asynchronous request queue for I/O scheduling
- Handles interrupts to coordinate request completion
- Supports LBA addressing for sector access
- Performs partition parsing for disk layout management

2. System Overview

2.1 System Architecture

The system architecture is shown in Figure 1. It is organized into three layers:

- **File System Layer** – provides system calls (`sys_open`, `sys_mkdir`, `sys_read`, `sys_write`, `sys_close`, `sys_delete`, `sys_flush`).
- **Block Cache Layer** – manages an in-memory cache of 4 KB data blocks using LRU replacement and write-back.
- **Disk Driver Layer** – handles asynchronous I/O requests, hardware interaction, and sector-level reads/writes.

Data Flow

- Hard-disk data are organized into 4 KB blocks.
- File system calls (e.g., `read()`, `write()`) are served by the File System Layer (via `sys_rdwt()`), mediated by the Block Cache Layer.
- On a cache hit, data are returned immediately.
- On a cache miss, a disk I/O request is enqueued in the Disk Driver Layer, and the calling process yields the CPU (state → Blocked).

Disk requests are serviced by `sys_hd_routine()`, repeatedly invoked by the user-level task (`iotask.c`).

- The I/O task dequeues requests and performs block reads/writes via sector-level HD functions asynchronously.
- The task yields the CPU and is reawakened by the HD interrupt handler once the request completes.
- Finally, the I/O task resumes the calling process, delivering the requested data to the user.

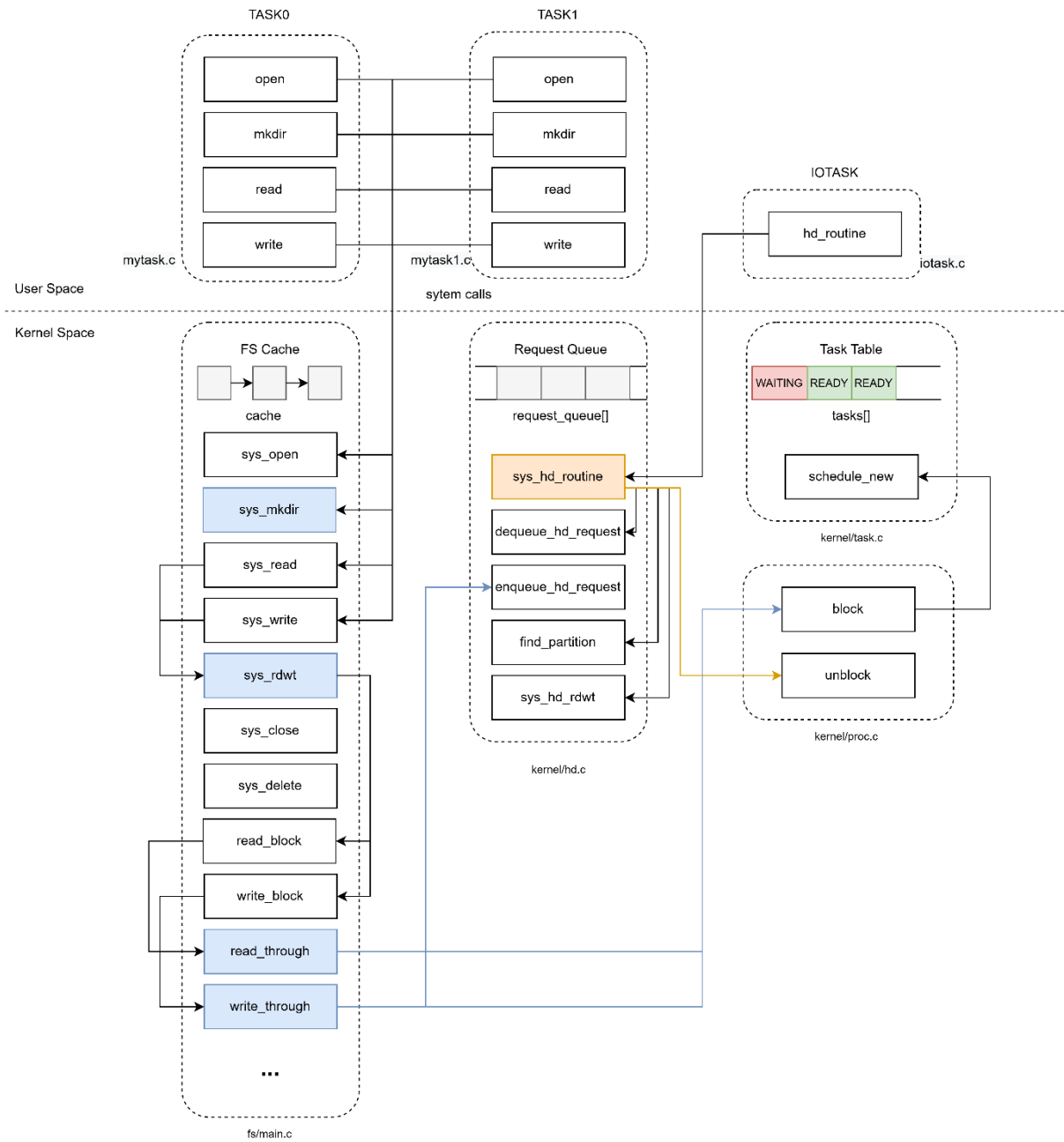


Figure 1. System architecture.

2.2 Module Breakdown

Module	File	Description
File System Core	<code>fs/main.c</code>	Superblock, inode, directory entry management, path resolution, file R/W with extension
Block Cache	<code>fs/main.c</code>	Cache allocation, lookup, LRU eviction, dirty-block synchronization
Disk Driver	<code>kernel/hd.c</code>	Disk initialization, partition table parsing, request queue management, interrupt handling, PIO
System Call Interface	<code>fs/main.c</code>	Entry points for open, read, write, close, delete, flush

3. Key Data Structure

3.1 File System

- **Superblock** (struct super_block): magic number, total inodes, total blocks, block size, inode size, directory entry size, root inode, bitmap positions, etc.

```
struct super_block {  
    u32 magic;      /**< Magic number */  
    u32 nr_inodes;  /**< How many inodes */  
    u32 nr_blocks;  /**< How many blocks (including bit maps) */  
    u32 nr_imap_blocks; /**< How many inode-map blocks */  
    u32 nr_bmap_blocks; /**< How many block-map blocks */  
    u32 first_data_block; /**< Number of the 1st data block */  
    u32 nr_inode_blocks; /**< How many inode blocks */  
    u32 root_inode;   /**< Inode nr of root directory */  
    u32 inode_size;   /**< INODE_SIZE */  
    u32 block_size;    /* 块大小（字节） */  
    u32 inode_isize_off; /**< Offset of `struct inode::i_size' */  
    u32 inode_start_off; /**< Offset of `struct inode::i_start_sect' */  
    u32 dir_ent_size;  /**< DIR_ENTRY_SIZE */  
    u32 dir_ent_inode_off; /**< Offset of `struct dir_entry::inode_nr' */  
    u32 dir_ent_fname_off; /**< Offset of `struct dir_entry::name' */  
    u8 _unused[4]; /**< Stuff for alignment */  
  
    /*  
     * the following item(s) are only present in memory  
     */  
    int sb_dev;  /**< the super block's home device */  
};
```

- **Inode** (struct inode): mode, size, direct/indirect block pointers, reference count, inode number.

```
/**
```

```

* @struct inode
* @brief i-node
* \b NOTE: Remember to change INODE_SIZE if the members are changed
*/

struct inode {
    u32 i_mode;    /**< Access mode. Unused currently */
    u32 i_size;    /**< File size */
    u32 i_start_block; /**< The first block of the data */
    u32 i_nr_blocks; /**< How many blocks the file occupies */
    u32 i_direct[INODE_DIRECT_COUNT]; /* 直接块指针 */
    u32 i_indirect;    /* 间接块指针 */
    u8 _unused[4]; /**< Stuff for alignment */

    /* the following items are only present in memory */
    //int i_dev;

    int i_cnt;    /**< How many procs share this inode */
    int i_num;    /**< inode nr. */
};

```

- **Directory Entry** (struct dir_entry): inode number and name (max 256 chars).

```

/**
* @struct dir_entry
* @brief Directory Entry
*/

struct dir_entry {
    int inode_nr;    /**< inode nr. */
    char name[MAX_FILENAME_LEN]; /**< Filename */
};

```

- **Inode cache** (`inode_table[]`): caches up to `NR_INODE` inodes in memory; each has a reference count.

```
EXTERN struct inode    inode_table[NR_INODE];
```

- **Global file descriptor table** (`f_desc_table[]`): holds inode pointer, access mode, current position for open files.

```
EXTERN struct file_desc f_desc_table[NR_FILE_DESC];
```

- **Per-process file descriptor array** (`current->filp[]`): points to entries in the global table.

3.2 Block Cache

- **Cache block** (`struct cache_block`): block number, data (4 KB), dirty flag, LRU pointers, access time.

```
/* 缓存块结构 */

struct cache_block {
    u32 block_nr;      /* 块号, -1 表示空闲 */
    u8 data[BLOCK_SIZE]; /* 数据 */
    int dirty;         /* 脏标志 */
    u32 access_time;   /* 访问时间 (用于 LRU) */
    struct cache_block *prev, *next; /* LRU 链表指针 */
};
```

- **LRU list**: `lru_head` and `lru_tail` maintain the least-recently-used order.

```
/* 全局变量 */

static struct cache_block* cache;      /* 缓存数组, 通过 kmalloc 分配 */
static struct cache_block* lru_head;   /* LRU 链表头 */
static struct cache_block* lru_tail;   /* LRU 链表尾 */
static int cache_initialized = 0;
static u32 cache_time_counter = 0;     /* 简单的时间计数器 */
```

3.3 Disk Driver

- **Request structure** (struct `hd_request`): PID, I/O type (read/write), block number, byte count, buffer.

```
/* 硬盘请求结构（以 4KB 块为单位） */
```

```
struct hd_request {
    int pid;          /* 请求进程的 PID */
    int io_type;      /* DEV_READ 或 DEV_WRITE */
    int block_nr;     /* block number */
    int bytes;        /* 字节数（应为 4KB 的倍数） */
    void* buf;        /* 用户缓冲区 */
};
```

- **Request queue**: circular queue `request_queue[]` of size `REQ_QUEUE_SIZE`.

```
/* FIFO 队列（环形队列） */
```

```
#define REQ_QUEUE_SIZE 32      /* 队列容量 */

static struct hd_request request_queue[REQ_QUEUE_SIZE];
static int queue_head = 0;
static int queue_tail = 0;
```

- **Partition info** (struct `hd_info`): base sector and size for primary and logical partitions.

```
/* main drive struct, one entry per drive */
```

```
struct hd_info
{
    /* int    cylinders; */
    /* int    heads; */
    /* int    sectors; */
    /* int    precomp; */
    /* int    lzone; */
    /* int    ctl; */

    int    open_cnt;

    struct part_info primary[NR_PRIM_PER_DRIVE];
    struct part_info logical[NR_SUB_PER_DRIVE];
};
```

4. Implementation Details

4.1 File System Initialization

`sys_init_fs` performs:

1. Clear file descriptor and inode tables.
2. Initialize block cache (`cache_init`).
3. Call `mkfs` to format the disk: write superblock, inode bitmap, block bitmap, root inode, and root directory data.
4. Read superblock into memory (`read_super_block`).
5. Read root inode into cache.

4.2 Path Resolution and Search

`search_file` parses absolute paths separated by `/`:

- Start from root inode, call `find_in_dir` for each component.
 - Return -1 if an intermediate directory is missing.
 - Return 0 if the file does not exist but the parent directory exists.
 - Return the inode number (>0) on success.
- `strip_path` extracts the parent directory path and filename, returning the parent directory inode (from cache or disk).

4.3 File Open (`sys_open`)

Logic flow:

- Call `search_file` to get the target inode number (distinguish invalid path, not found, found).
- Find free slots in the per-process file descriptor array and the global file descriptor table.
- Based on flags:
 - `O_CREAT`: if file does not exist, call `create_file`; if exists, return error.
 - Otherwise: if file exists, get inode via `read_inode`; else error.
 - Link the process file descriptor to the global table entry, set initial position to 0.
 -

4.4 File Read/Write (`sys_rdwrt`)

- Supports regular files (special branch calling `sys_hd_rdwrt`).
- **Write-ahead allocation**: before writing, compute needed logical blocks, call `alloc_block_for_inode` to allocate missing blocks (direct + indirect).
- For each logical block, obtain physical block via `get_block_nr`.
- Read the block into a temporary buffer (through cache or direct I/O).
- Copy data to/from user space using `phys_copy` (address translation via `va2la`).
- If write, mark block dirty and write back.
- Update file position and inode size if extended; write inode to disk.
- Read operations fill holes with zeros.

4.5 File Delete (`sys_delete`)

- Get inode number via `search_file`; check existence.
- Use `strip_path` to get parent directory inode and filename.
- Scan parent directory blocks to find the matching directory entry, clear its `inode_nr`, and write back.
- Read file inode, free all data blocks (update block bitmap via `free_block`).
- Free the inode number (update inode bitmap via `free_inode`).
- Remove inode from cache (set reference count to 0).
- Optionally sync parent directory inode.

4.6 Block Cache Mechanism

- **Read path:** `read_block` first searches the cache. On hit, copies data; on miss, calls `read_through` (synchronous I/O) and adds the block to the cache (clean).
- **Write path:** `write_block` adds the block to the cache (dirty). Actual disk write happens during cache eviction or explicit sync.
- **Eviction:** `cache_add`: when cache is full selects the LRU tail. If that block is dirty, writes it back via `write_through`, then reuses the slot.
- **Sync:** `cache_sync_all` iterates over all cache blocks and writes dirty ones via `write_through`.

4.7 Disk Driver

- **Initialization:** `sys_init_hd` sets IRQ handler and enables interrupts.
- **Partition detection:** `sys_hd_open` triggers `hd_identify` to get disk info, then `partition` parses primary and extended partition tables, populating `hd_info`.
- **Request queue:** `enqueue_hd_request` and `dequeue_hd_request` manage a circular queue. `sys_hd_routine` (background routine) processes one request at a time: translates the absolute block number to a partition-local byte offset using `find_partition`, then calls `sys_hd_rdwt` for actual sector I/O.
- **Low-level I/O:** `sys_hd_rdwt` builds ATA commands (LBA28), uses PIO to transfer data, and waits for interrupt completion (`interrupt_wait` blocks the process and is woken by the interrupt handler).
- **Interrupt handler:** `hd_handler` reads the status register and wakes the waiting process.